

Part 1 Pattern (PATTERN.v):

1. 測資生成與限制條件

為了符合 SPEC 中對於記憶體存取位址的嚴格限制，我在 `generate_instructions_and_data` task 中實作了動態的位址偏移追蹤。當生成 Load/Store 指令時，會根據當前暫存器的值（`sim_r1` / `sim_r2`）去動態計算 Immediate，確保最終的 Data DRAM 存取位址絕對落在前一次存取的正負 200 範圍內。此外，對於 Jump 與 Branch 指令，我也加入了邊界檢查，防止 `next_pc` 溢出 0x1FFF 的 Instruction DRAM 範圍，並在指令的前期隨機混入向前與向後的跳躍，以產生具備執行迴圈能力以及出現 Data Hazard 情況的測資。

2. AXI Snoopy 以及 Golden Model 建立

在每個 Cycle，Pattern 會嚴密監控 AXI4 介面的 Handshake，包含防範 VALID 掉落、READY Timeout 甚至最後的 AXI Flush 死鎖檢查。

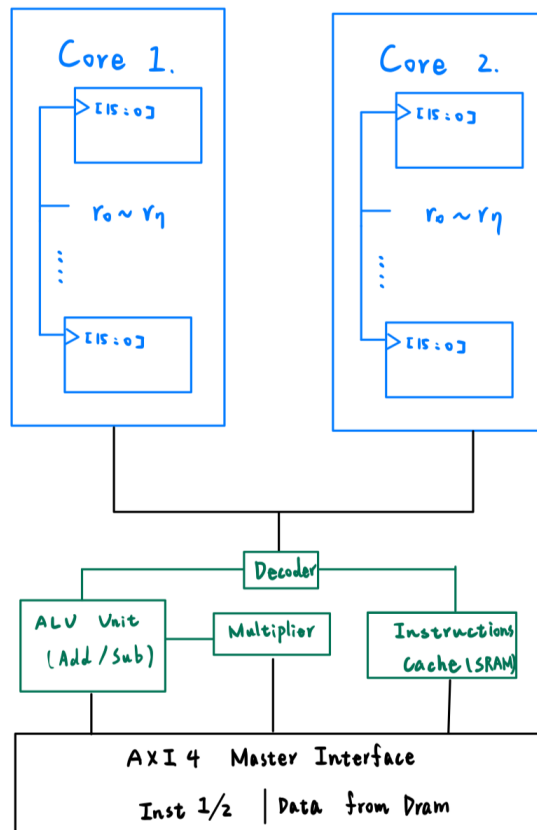
在 simulation 時，只要硬體的 `stall_1` 或 `stall_2` 被拉低，Pattern 就會立刻比對對應 Core 的 8 個暫存器狀態；同時實作了嚴格的「50-IC Checkpoint」，當雙核中較慢的一方也達到 50 的倍數個指令時，立即去看看 Data DRAM 的 Snapshot，確保雙核的 Store 執行順序與覆寫邏輯完全與 pattern 相同。

3. 測資的 Halt Condition 與邊界判定

在指令生成上，我在實作時除了確保 Data DRAM 的存取落在 ± 200 bytes 內，也嚴格遵守了助教在討論區的釋疑：確保所有 Jump 與 Branch 的目標位址必定為偶數，避免硬體產生 Unaligned Access 的錯誤。

在 Halt Condition 的控制中，Pattern 會精確計算雙核執行的指令總數。當雙核皆執行完相同數量的指令，且抵達 Instruction DRAM 的最後一個有效位址時，Pattern 會啟動 Final AXI Flush 檢查，確保所有的 AXI Handshake 皆已乾淨收尾，徹底杜絕了 Deadlock 的發生。

Part 2 Design (DCCPU.v)



1. 面積為主體進行優化

Design 在設計時，因為知道 Performance 的計算是以「面積平方 (Area^2)」為權重，所以一開始就打算採取壓縮面積的方式，盡可能減小面積。

單一 SRAM 快取架構：

我放棄了為兩個 Core 各自配置獨立 SRAM 的常規做法，而是宣告了一顆 64x16 的 SRAM64X16。在設計上是用 Direct-Mapped 的概念，透過 $\text{pc}[5:4]$ 加上 burst_cnt 作為位址，巧妙地將雙核的 Instruction 存取映射到同一顆物理 SRAM 上。這不僅省下了可觀的面積，也完美配合了 AXI 的 INCR Burst 特性。

分時多工 Datapath：

除了 Cache，包含 ALU 以及 Multiplier 也都是雙核共用。乘法器我採用了 Radix-2 Booth Algorithm 的循序架構，將其包在 S_MULT 的狀態內花費 16 個 cycle 完成，這在大幅縮減 Combinational Logic 面積的同時，也優化了 Critical Path。ALU 方面，我將減法與比較 (SLT) 全數透過 XOR 閘與 Carry-in 整合進同一個加法器中，做到面積最小化。

2. 解決 Data Hazard 的 FSM

為了避免雙核共用 Data DRAM 所帶來的複雜 Hazard 問題（尤其是 SPEC 規定的 Store-before-Load Priority），我在狀態機中設計了交替執行的機制。在 S_DEC_ORDER 狀態下，硬體會預先解碼兩邊的指令：

```
if (!halt_1 && !halt_2) begin
    if ((inst_2_reg[15:13] == 3'b101) && (inst_1_reg[15:13] != 3'b101)) begin
        first_core <= 1'b1; cur_core <= 1'b1;
    end else begin
        first_core <= 1'b0; cur_core <= 1'b0;
    end
end
```

當發現同一個 Instruction Count 下，Core 2 是 Store 而 Core 1 是 Load 時，FSM 會強制讓 Core 2 先進入執行階段寫入 Data DRAM，隨後才輪到 Core 1 進行讀取。這樣的設計無須外掛龐大的 Forwarding Path 或 Arbiter，就自然而然地滿足了 SPEC 嚴格的先後順序要求，進一步達成了面積壓縮的目標。

3. Stall 訊號的精確時序控制

為了配合 Pattern 嚴格的檢查機制（Stall 為 Low 時暫存器值不得變動），我在 FSM 的 Write Back 階段做了 Phase 的切割。來去確保無論指令執行的快或慢，stall_1 與 stall_2 都只會在暫存器資料寫入穩定後的同一個 Cycle 拉低一週期，隨後立刻拉高。此外，針對雙核執行進度不一的狀況，當其中一顆 Core 提早抵達 Halt condition 時，其對應的 Stall 訊號會被強制鎖死在 HIGH，完美避免了 Pattern 在結束前進行多餘的暫存器採樣而導致 Latency 違規。

4. Performance 與 Latency 的權衡演進

而在優化的過程中也是以「減小面積」這個方向去進行，硬體方面能共用的共用、能夠分時多工的點也儘量做到，可以看到面積一路降低。

在前期版本（v1 到 v3），我成功將 Area 從近 50 萬大幅縮減到 22 萬，在 Performance 分數上確實取得不錯的效果。但後面與其他人比較 Latency 後，發現因為高度的序列化（Serial）設計，導致我的總 Cycle 數幾乎是其他人的數倍以上。所以後期版本（v10 到 v12）我轉向更積極地壓低 Cycle Time，將時脈從 4.5ns 逐步逼近至 3.7ns。因為整體電路架構本身已經偏向極致精簡，所以後期的優化次數並沒有到很多，主要專注於 Critical Path 的時序收斂。最終是落在 Area 146k 與 CT 3.7ns 的 performance。

VERSION	AREA	CT	Latency	PPA
v1	493,406	4.5	23,238	2.546E+16
v2	223,311	12.0	13,984	8.368E+15
v3	246,508	9.0	13,984	7.648E+15
v5	265,635	5.5	11,986	4.652E+15
v6	236,132	6.5	11,773	4.267E+15
v6	254,514	5.3	11,773	4.042E+15
v7	243,736	5.5	11,773	3.847E+15
v7_1	244,737	5.5	11,773	3.878E+15
v8	157,824	5.5	22,879	3.134E+15
	140,587	6.5	22,879	2.939E+15
	137,118	7	150	1.974E+13
	145,291	4	150	1.267E+13
v10	131,845	4	150	1.043E+13
	130,944	4.5	150	1.157E+13
	132,840	3.7	150	9.794E+12
v11	138,600	3.7	150	1.066E+13
	132,274	4	150	1.050E+13
	204,228	5	110	2.294E+13
v12	146,169	3.7	117	9.249E+12
	145,151	4	116	9.776E+12

Part 3 心得

雖然在題目公布前就有預先想到題目可能跟 CPU 有相關，但沒想到是這種雙核心的形式，著實有些震驚，並且訊號線其實蠻多的，畢竟也是 AXI-4 的 interface 所以難度肯定不低，而也確實煞費苦心，中間還遇到期中考、OT，以及幾個 Lab，加上還要吸收課堂上的東西確實有些困難，對於題目的理解也非常緩慢，還好助教都非常有耐心的回答我們在 DC 上提出的問題，這邊說聲辛苦了，學期過了一半了期許自己能撐住，也感謝助教一路的相伴，大感恩。